

AWS CLOUD INFRASTRUCTURE PROJECT

Hosting a Website on Ubuntu EC2 Instances using a Launch Template and Auto Scaling Group

A hands-on guide to high-availability, self-healing web hosting on Amazon EC2

Parameter	Value
Cloud Provider	Amazon Web Services (AWS)
Operating System	Ubuntu Server 26.04 LTS
Web Server	Apache2 (HTTP)
Core Services Used	VPC, Subnets, Internet Gateway, Route Table, Launch Template, Auto Scaling Group, Security Groups, EC2
Region	US East (N. Virginia) — us-east-1

1. Objective

The objective of this project is to design and deploy a fault-tolerant, self-healing website hosting environment on AWS. Instead of manually launching and configuring a single EC2 instance, the deployment uses a Launch Template to standardize instance configuration and an Auto Scaling Group (ASG) to automatically launch, monitor, and replace EC2 instances running Ubuntu Server. The result is a website that remains available even if an individual instance fails, and that can automatically scale out to handle increased traffic.

By the end of this project, the following outcomes are achieved:

- A custom Virtual Private Cloud (VPC) with public subnets across two Availability Zones.
- Internet connectivity enabled through an Internet Gateway and a public route table.
- A reusable Launch Template that defines the AMI, instance type, key pair, and a bootstrap (user data) script.
- An Auto Scaling Group that uses the Launch Template to automatically launch and maintain a desired number of healthy EC2 instances.
- A running Apache web server, installed automatically at boot time, serving a sample web page over HTTP.

2. Theory

2.1 Amazon VPC (Virtual Private Cloud)

A VPC is a logically isolated section of the AWS cloud where resources such as EC2 instances are launched. It gives full control over the networking environment, including the IP address range, subnets, route tables, and gateways. Every VPC is defined by a CIDR block (e.g. 192.168.1.0/24) that determines the range of private IP addresses available inside it.

2.2 Subnets and Availability Zones

A subnet is a smaller segment of a VPC's IP address range, tied to a single Availability Zone (AZ). A Public Subnet is one whose route table sends internet-bound traffic (0.0.0.0/0) to an Internet Gateway, allowing instances inside it to be reached from the internet. Spreading public subnets across two or more Availability Zones is a core AWS best practice — it ensures that the failure of a single data center (AZ) does not take the whole application offline, which is essential for the Auto Scaling Group used later in this project.

2.3 Internet Gateway (IGW)

An Internet Gateway is a horizontally scaled, redundant VPC component that allows communication between instances in a VPC and the internet. It performs network address translation (NAT) for instances that have a public IPv4 address and must be explicitly attached to a VPC before it can be used.

2.4 Route Tables

A route table contains a set of rules, called routes, that determine where network traffic from a subnet is directed. To make a subnet "public", a route is added that sends all traffic not destined for the local VPC CIDR (i.e.

0.0.0.0/0) to the Internet Gateway. The route table is then associated with the subnet(s) that should have internet access.

2.5 Launch Templates

A Launch Template captures all the configuration needed to launch an EC2 instance — the Amazon Machine Image (AMI), instance type, key pair, security groups, storage, and user data script — as a single reusable, versioned resource. Instead of configuring each EC2 instance by hand, the Auto Scaling Group refers to the Launch Template and uses it as a blueprint every time a new instance needs to be launched. AWS now recommends Launch Templates over the older Launch Configurations, since templates support versioning and newer EC2 features.

2.6 User Data (Bootstrap Script)

User data is a script that EC2 runs automatically the very first time an instance boots. It is used here to bootstrap the web server without any manual intervention: updating the package index, installing the Apache2 web server, starting and enabling the service, and writing a simple HTML page. Because this script lives inside the Launch Template, every instance the Auto Scaling Group creates configures itself identically and becomes a working web server within moments of launching.

2.7 Auto Scaling Group (ASG)

An Auto Scaling Group is a collection of EC2 instances that AWS treats as a logical group for scaling and management. The ASG uses the Launch Template to launch new instances, continuously performs health checks, and automatically replaces any instance that becomes unhealthy or is terminated — this is what makes the deployment "self-healing". It also allows a Minimum, Desired, and Maximum instance count to be defined, and can later be attached to scaling policies (e.g. CPU-based target tracking) so that capacity grows and shrinks automatically with demand.

2.8 Security Groups

A security group acts as a virtual firewall attached to an EC2 instance's network interface, controlling inbound and outbound traffic at the instance level. For this project, the security group must allow inbound SSH (port 22) so the instance can be administered, and inbound HTTP (port 80) so the website is reachable from a browser.

3. Architecture Overview

The final architecture consists of a custom VPC containing two public subnets in different Availability Zones (us-east-1a and us-east-1b). Both subnets route internet-bound traffic through a single Internet Gateway via a shared public route table. A Launch Template defines how each Ubuntu EC2 instance should be configured and bootstrapped, and an Auto Scaling Group deploys instances from this template across both subnets, keeping two healthy Apache web servers running at all times.

Parameter	Value
VPC Name / CIDR	myvpc — 192.168.1.0/24
Public Subnet 1	public-subnet1 — 192.168.1.0/25 — AZ us-east-1a

Parameter	Value
Public Subnet 2	public-subnet2 — 192.168.1.128/25 — AZ us-east-1b
Internet Gateway	My-IGW, attached to myvpc
Route Table	Public-RT — 0.0.0.0/0 → My-IGW
Launch Template	newtemp — Ubuntu 26.04 LTS, t3.small, key pair test1
Auto Scaling Group	autosg — Desired: 2, Min: 1, Max: 10

4. Implementation Steps

Step 1: Create a VPC

- ▶ Navigate to the VPC console → click Create VPC.
- ▶ Select VPC only (not "VPC and more").
- ▶ Set Name tag: myvpc
- ▶ Set IPv4 CIDR block: 192.168.1.0/24
- ▶ Click Create VPC.

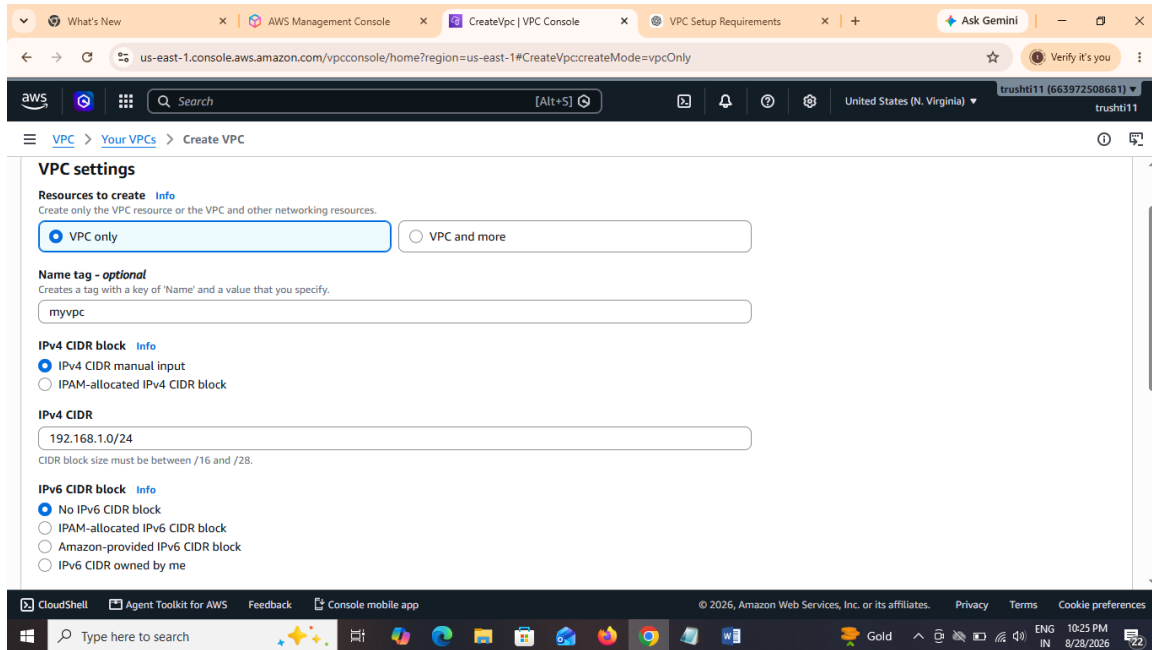


Figure 1 — Creating the custom VPC 'myvpc' with CIDR 192.168.1.0/24

Step 2: Create Two Public Subnets

- ▶ Go to Subnets → Create subnet, and select myvpc as the parent VPC.
- ▶ Create the first subnet: Name public-subnet1, Availability Zone us-east-1a, IPv4 CIDR block 192.168.1.0/25.

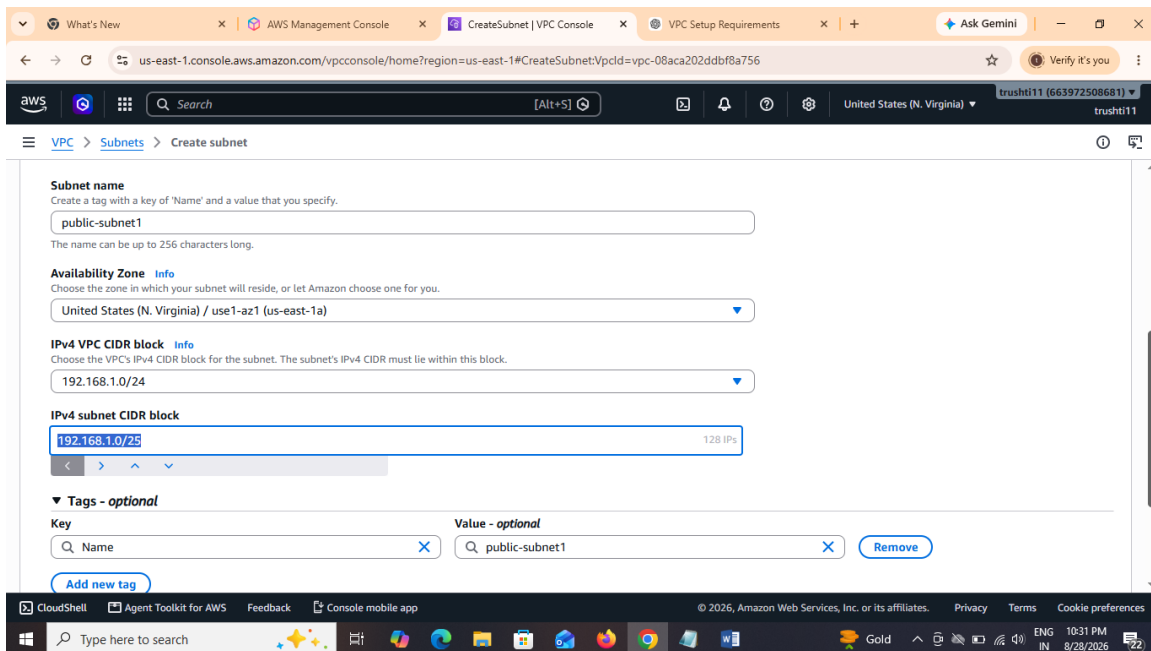


Figure 2 — Creating public-subnet1 in Availability Zone us-east-1a

- ▶ Repeat the process to create a second subnet: Name public-subnet2, Availability Zone us-east-1b, IPv4 CIDR block 192.168.1.128/25.

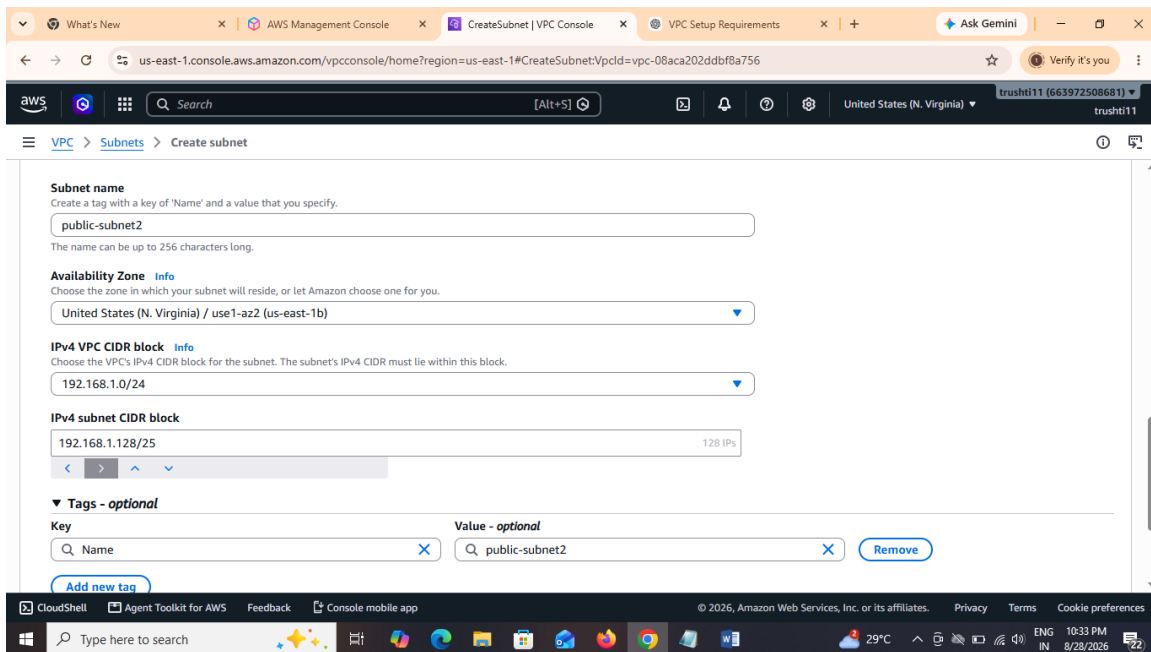


Figure 3 — Creating public-subnet2 in Availability Zone us-east-1b

Why two subnets?: Placing subnets in two different Availability Zones lets the Auto Scaling Group spread instances across separate data centers, so the website stays available even if one AZ has an outage.

Step 3: Enable Auto-Assign Public IP

- ▶ Select each public subnet → Actions → Edit subnet settings.
- ▶ Check Enable auto-assign public IPv4 address.

- Click Save.

This ensures every instance launched into these subnets automatically receives a public IPv4 address, so it can be reached directly from the internet.

Step 4: Create and Attach an Internet Gateway

- Go to Internet Gateways → Create internet gateway → Name: My-IGW → Create.

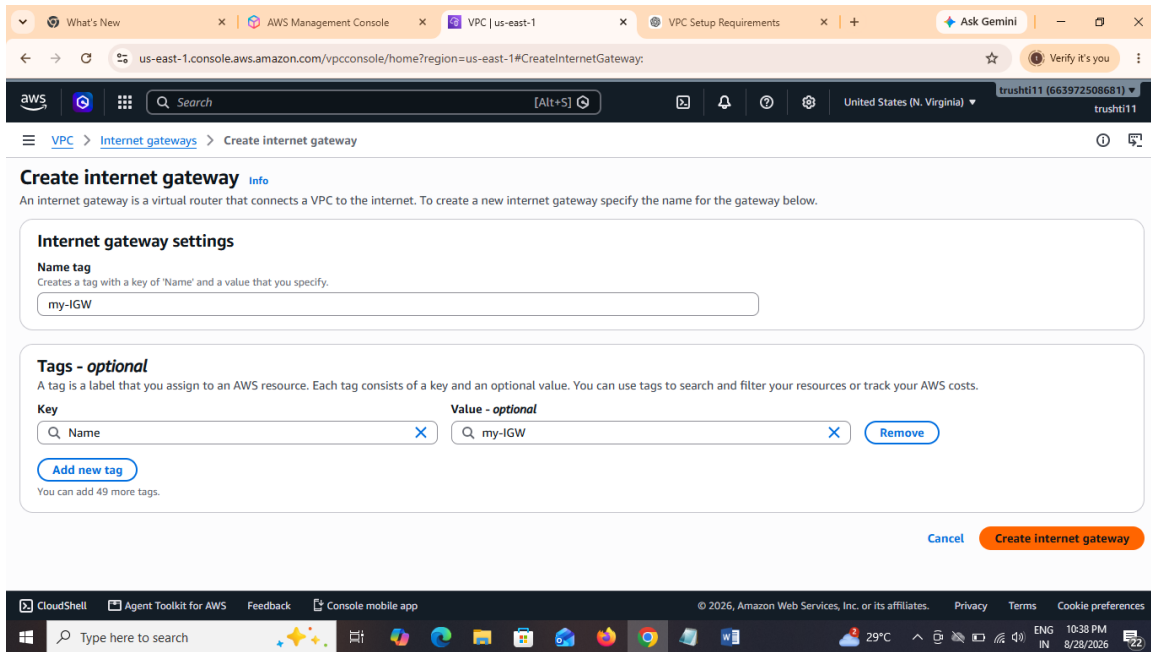


Figure 4 — Creating the Internet Gateway 'My-IGW'

- Select the newly created IGW → Actions → Attach to VPC → choose myvpc → Attach.

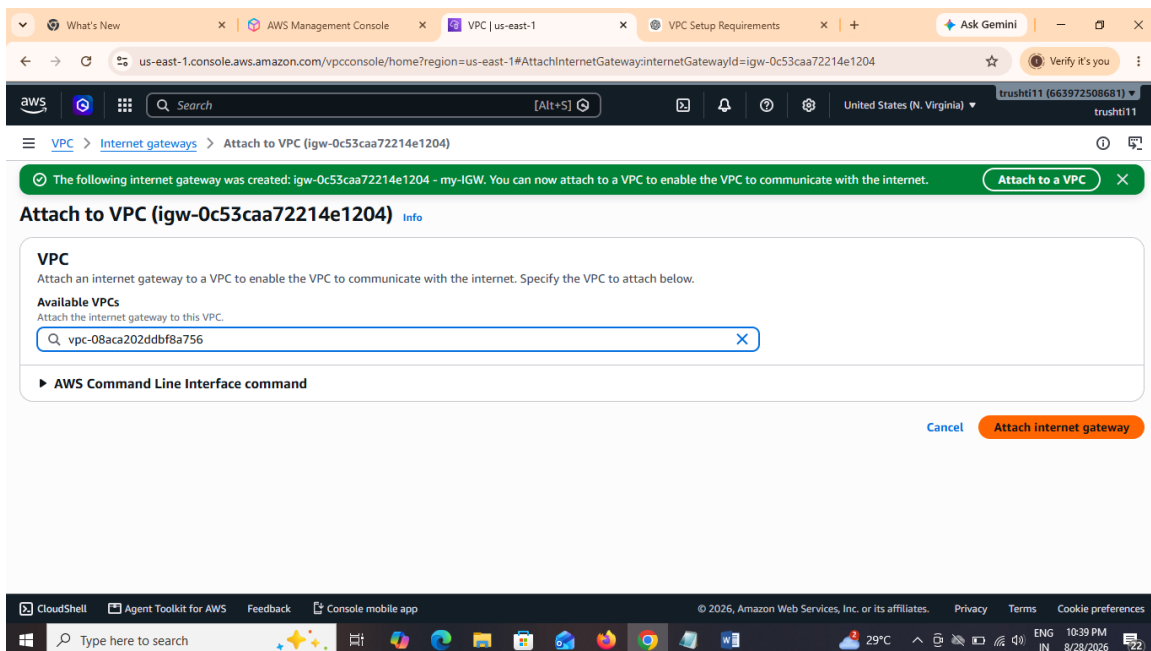


Figure 5 — Attaching My-IGW to myvpc

Step 5: Create a Public Route Table

- Go to Route Tables → Create route table → Name: Public-RT, VPC: myvpc → Create.

The screenshot shows the 'Create route table' page in the AWS Management Console. The 'Name' field is set to 'Public-RT' and the 'VPC' dropdown is set to 'vpc-08aca202ddb8a756 (myvpc)'. There are no tags added yet. The 'Create route table' button is visible at the bottom right.

Figure 6 — Creating the 'Public-RT' route table inside myvpc

- Open Public-RT → Routes tab → Edit routes → Add route.
- Set Destination: 0.0.0.0/0, Target: Internet Gateway (My-IGW) → Save routes.

The screenshot shows the 'Edit routes' page for the 'Public-RT' route table. The 'Destination' is set to '0.0.0.0/0' and the 'Target' is set to 'Internet Gateway'. The 'Status' is 'Active'. The 'Add route' button is visible at the bottom left.

Figure 7 — Adding a default route (0.0.0.0/0) pointing to My-IGW

Step 6: Associate the Route Table with the Public Subnets

- Open Public-RT → Subnet Associations tab → Edit subnet associations.
- Select both public-subnet1 and public-subnet2 → Save.

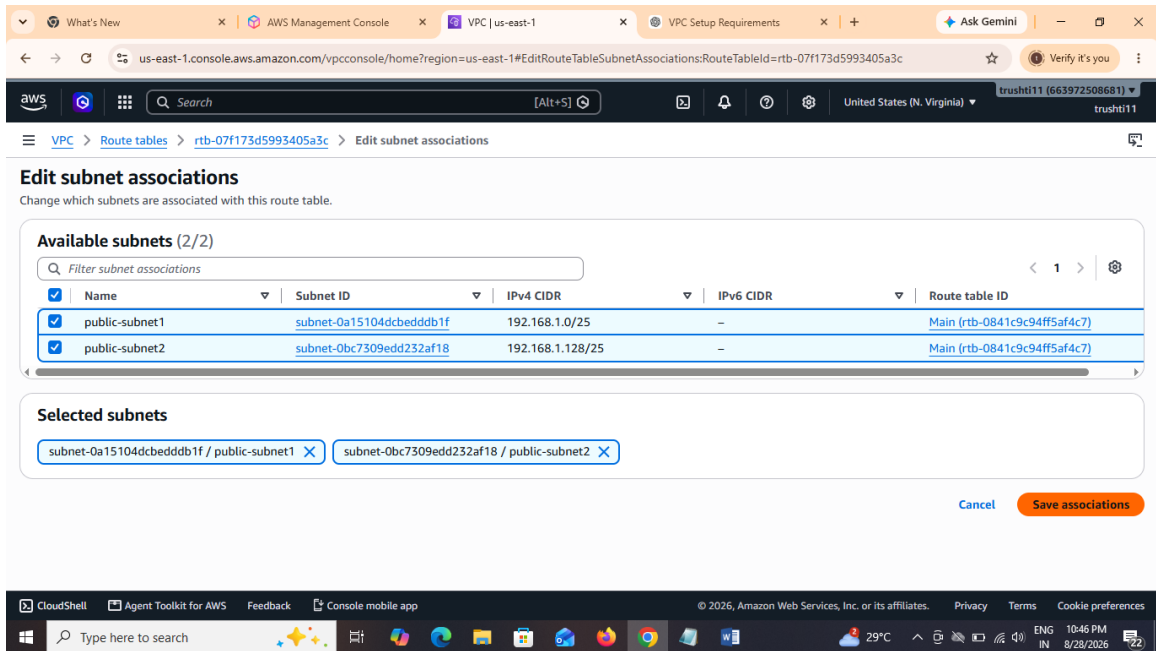


Figure 8 — Associating the public subnets with Public-RT

Once associated, both subnets can route traffic to and from the internet, completing the networking layer of the architecture.

Step 7: Create a Launch Template

- ▶ Go to EC2 → Launch Templates → Create launch template, and give it a name (e.g. newtemp).
- ▶ Under Application and OS Images (AMI), select Ubuntu — Ubuntu Server 26.04 LTS.

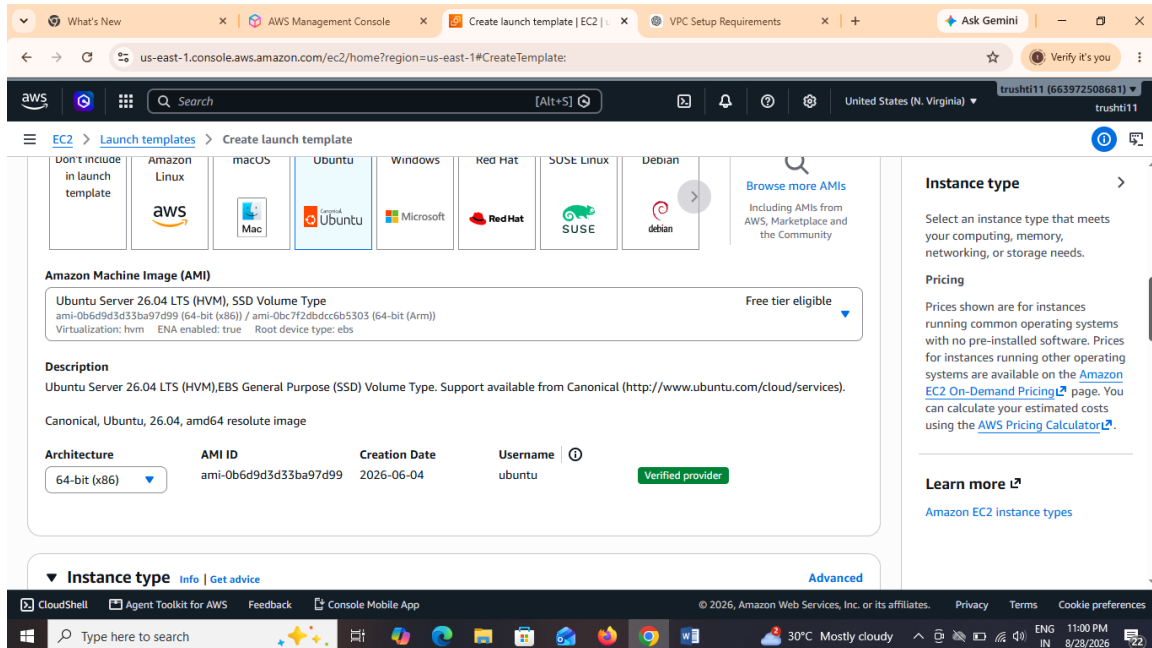


Figure 9 — Selecting the Ubuntu Server 26.04 LTS AMI

- ▶ Choose an Instance type (e.g. t3.small) and select an existing Key pair (e.g. test1) for SSH access.

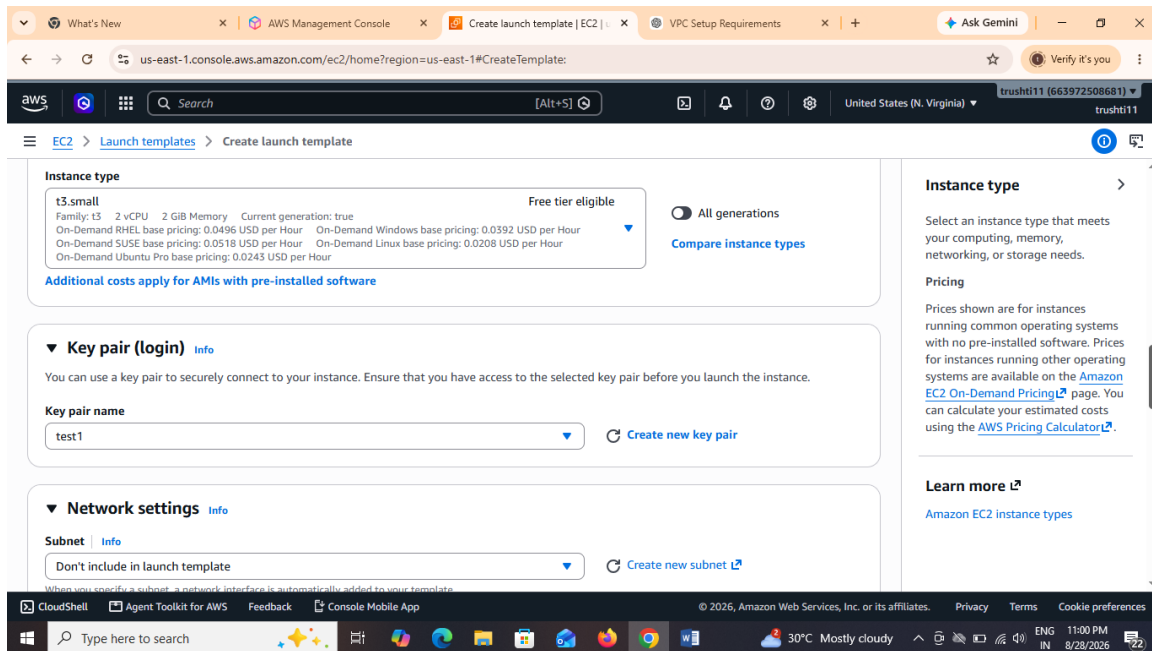


Figure 10 — Selecting instance type t3.small and key pair 'test1'

- ▶ Scroll to the User data field and paste a bootstrap script that installs and starts Apache automatically on first boot:

```
#!/bin/bash
```

```
# Update packages
apt-get update -y

# Install Apache web server
apt-get install -y apache2

# Start Apache
systemctl start apache2
systemctl enable apache2

# Create a simple webpage
echo "<h1>Hello from Ubuntu EC2!</h1>" > /var/www/html/index.html
```

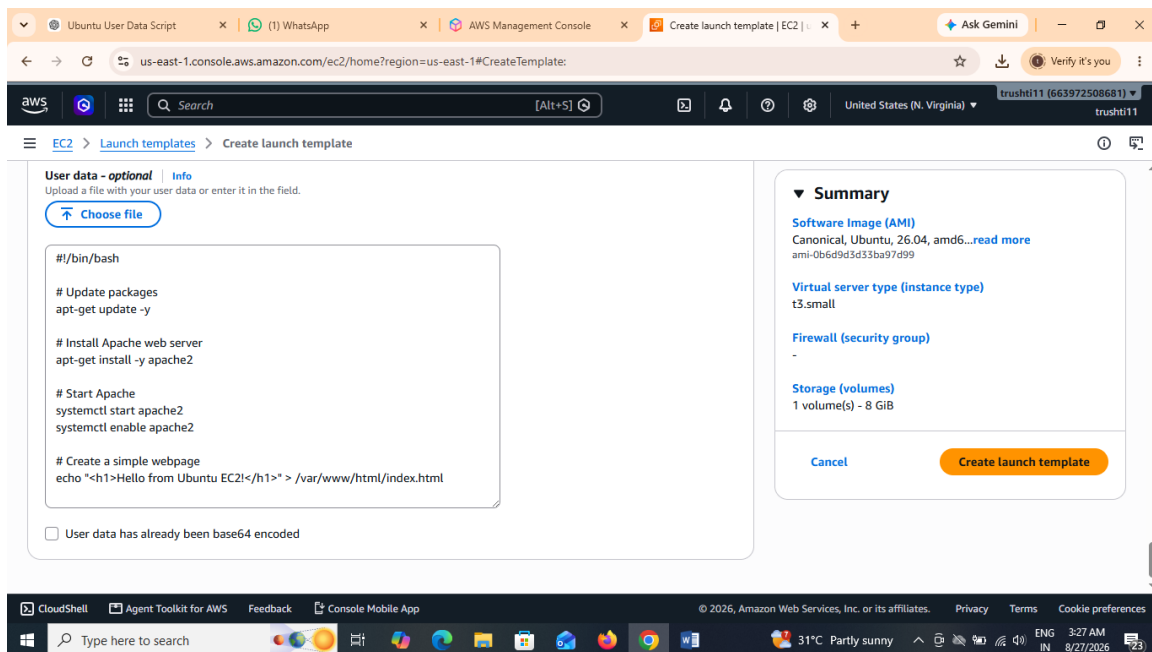


Figure 11 — User data script pasted into the launch template

- ▶ Leave network settings as "Don't include in launch template" (the subnet will instead be chosen when the Auto Scaling Group is created), and click Create launch template.

Step 8: Create the Auto Scaling Group

- ▶ Go to Auto Scaling Groups → Create Auto Scaling group.
- ▶ Enter a Name (e.g. autosc) and select the launch template created above (newtemp, Default version).

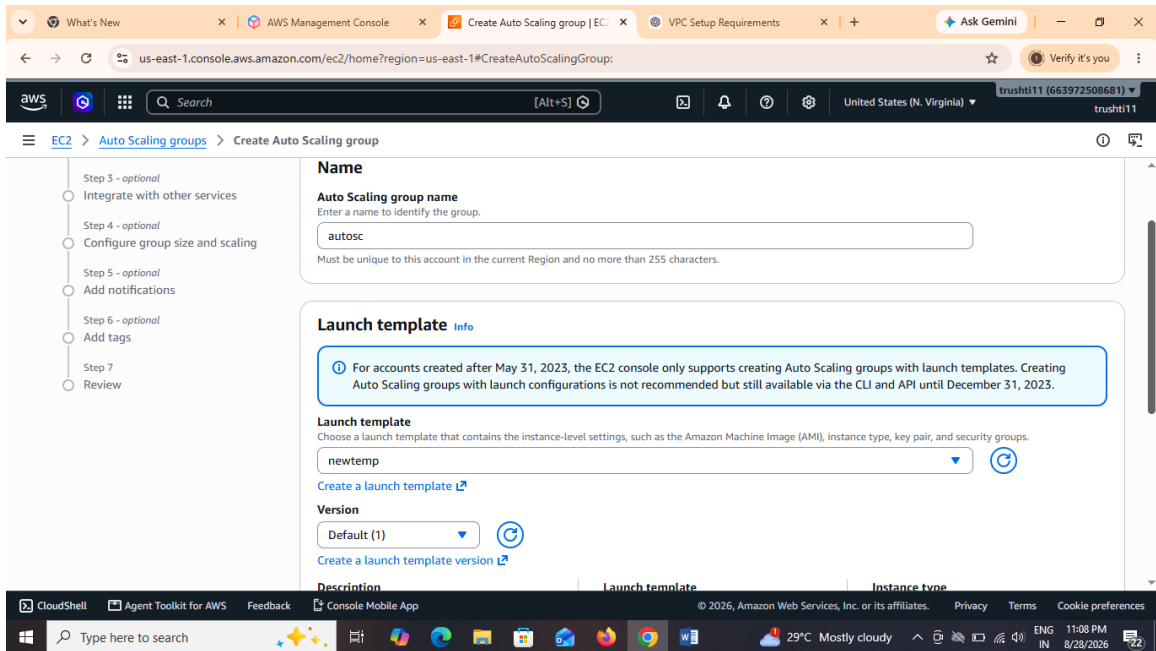


Figure 12 — Naming the Auto Scaling group and selecting the launch template

- Choose the VPC (myvpc) and select both Availability Zones / subnets (public-subnet1 and public-subnet2) so instances are spread across both AZs.

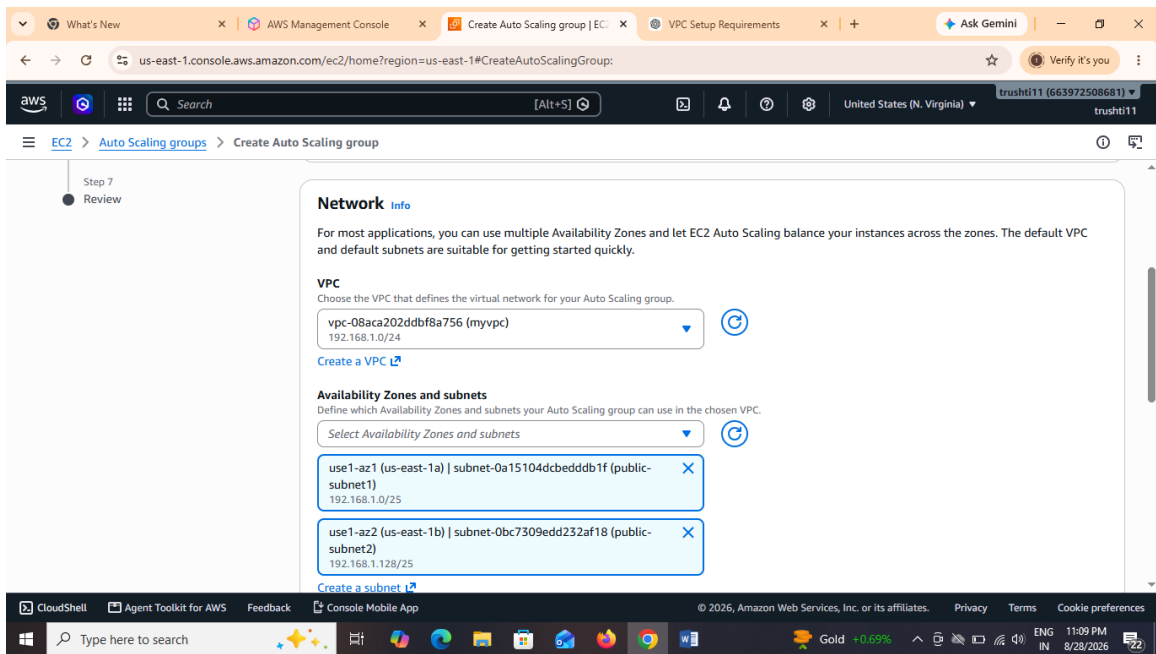


Figure 13 — Selecting myvpc and both public subnets for the Auto Scaling group

- On the Configure group size and scaling step, set Desired capacity: 2, Minimum: 1, Maximum: 10. For this project, No scaling policies is selected (capacity stays fixed at the desired count).

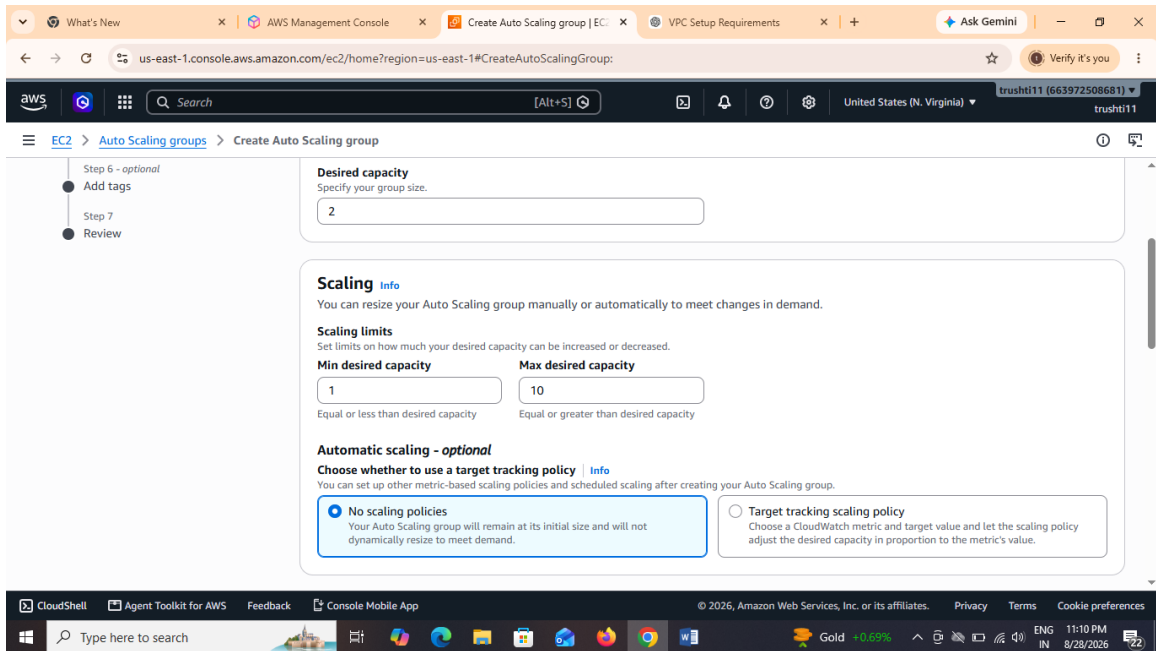


Figure 14 — Setting desired/min/max capacity for the Auto Scaling group

- Skip notifications and tags (optional), review the configuration summary, and click Create Auto Scaling group.

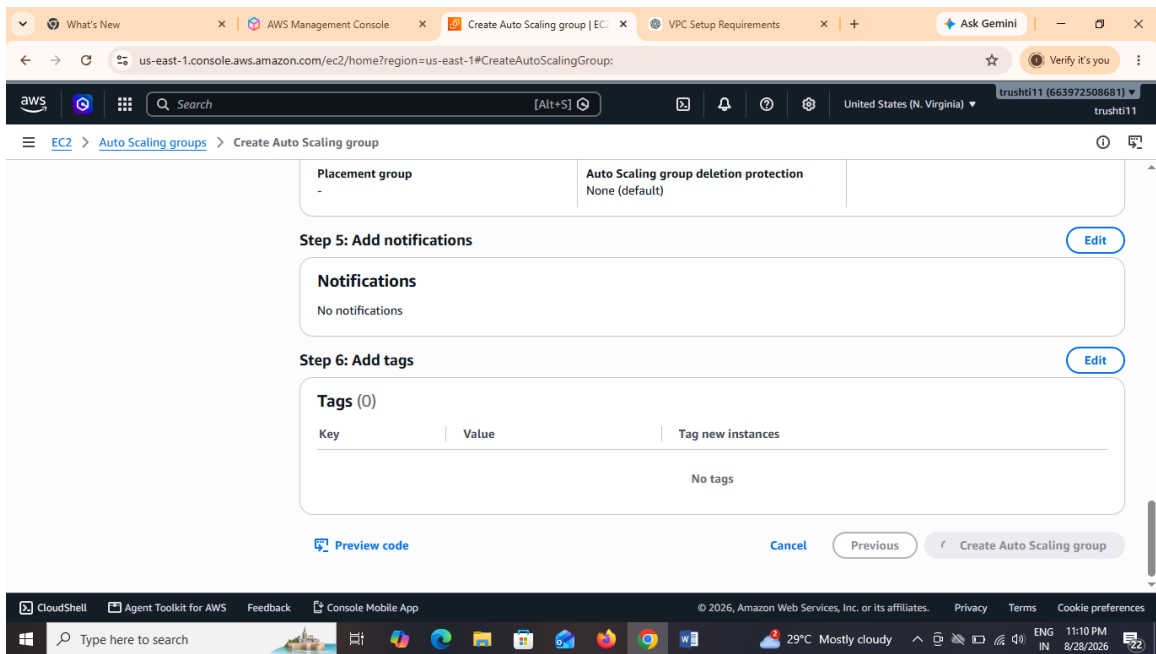


Figure 15 — Reviewing notifications and tags before creating the group

Step 9: Verify Instances and Configure the Security Group

After creation, the Auto Scaling Group's Activity tab shows its status changing until the target capacity is reached. Once complete, the EC2 Instances page shows two new instances launched automatically — one in each public subnet — without any manual EC2 launch step.

- ▶ Wait for at least one instance to pass its status/security checks.
- ▶ Open the instance's security group → Edit inbound rules, and add rules to allow SSH (port 22) and HTTP (port 80) from 0.0.0.0/0.

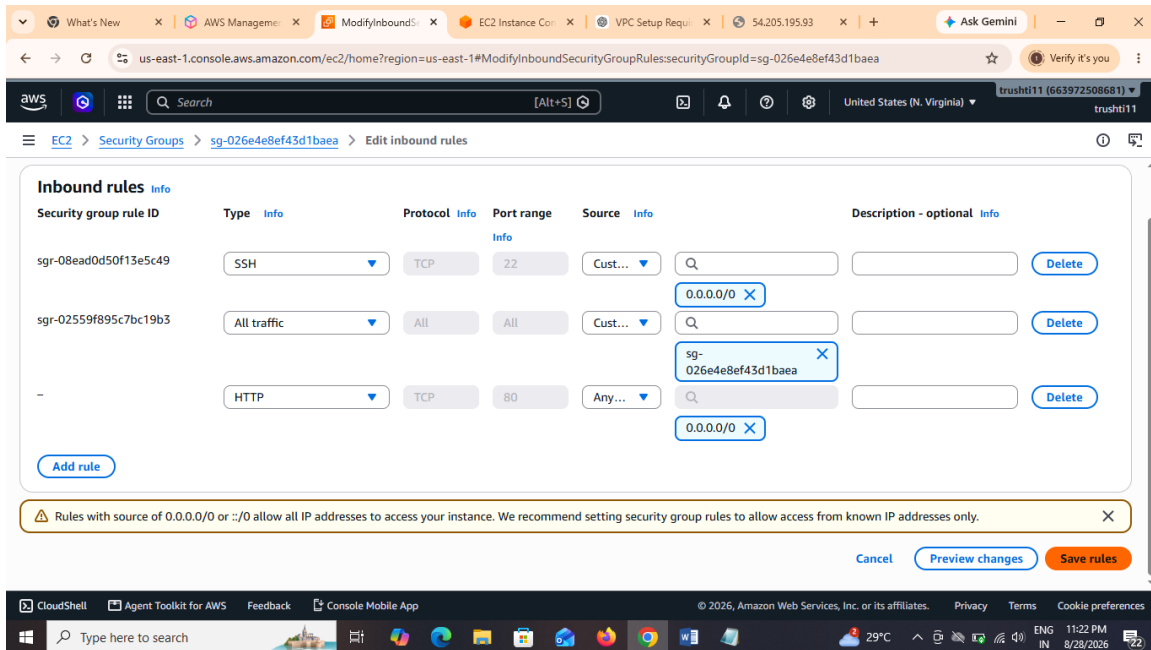


Figure 16 — Security group inbound rules allowing SSH (22) and HTTP (80)

Step 10: Connect to an Instance and Verify Apache

- ▶ Select an instance → Connect → EC2 Instance Connect (username: ubuntu) → Connect.

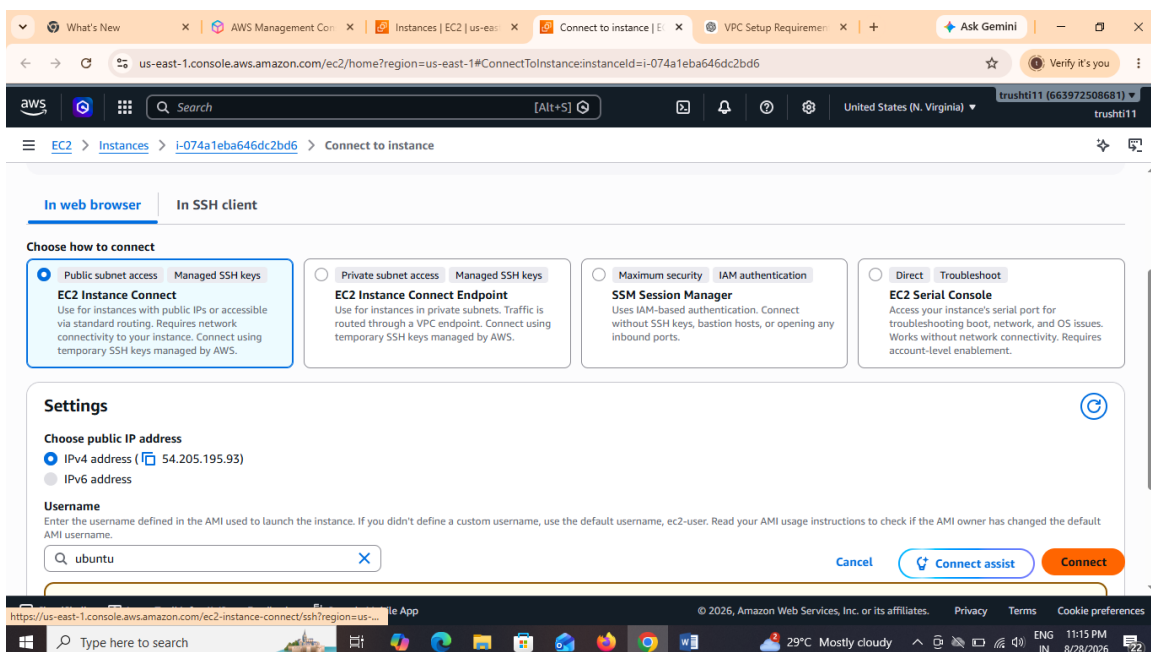


Figure 17 — Connecting to the Ubuntu instance via EC2 Instance Connect

- Once connected, verify the web server is running:

```
sudo systemctl status apache2
```

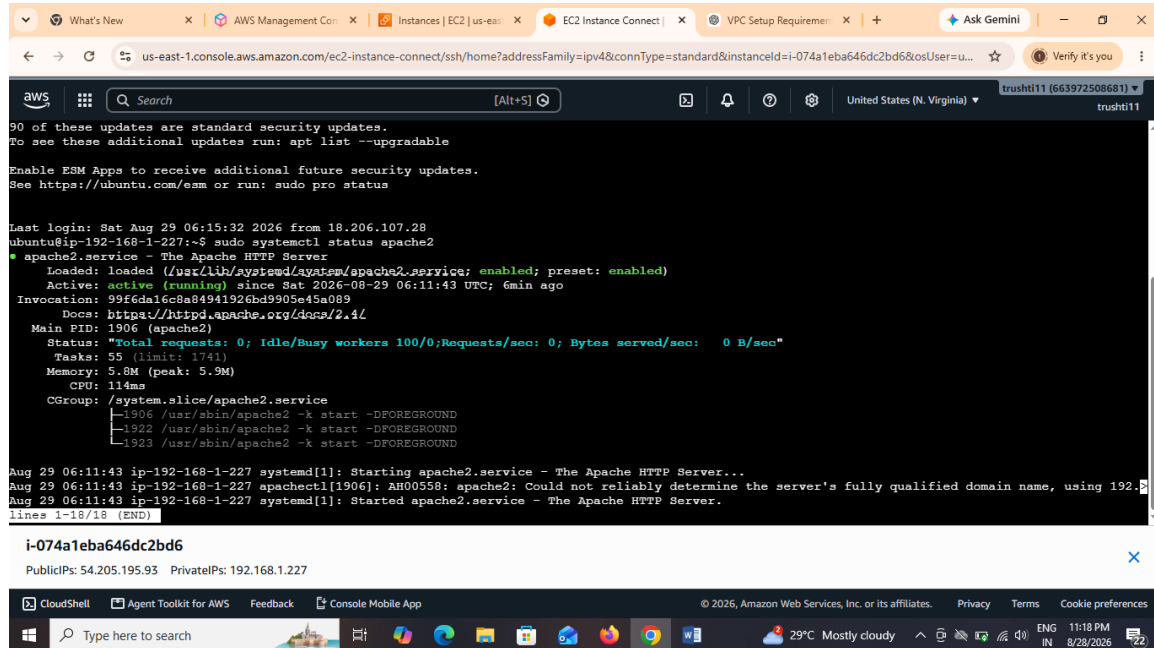


Figure 18 — 'apache2.service' shown as active (running) on the Ubuntu instance

The output confirms that `apache2.service` loaded successfully and is active (running) — proof that the user data script executed correctly when the instance booted, with no manual installation required.

Step 11: Test the Website

- Copy the instance's Public IPv4 address and open it in a web browser.

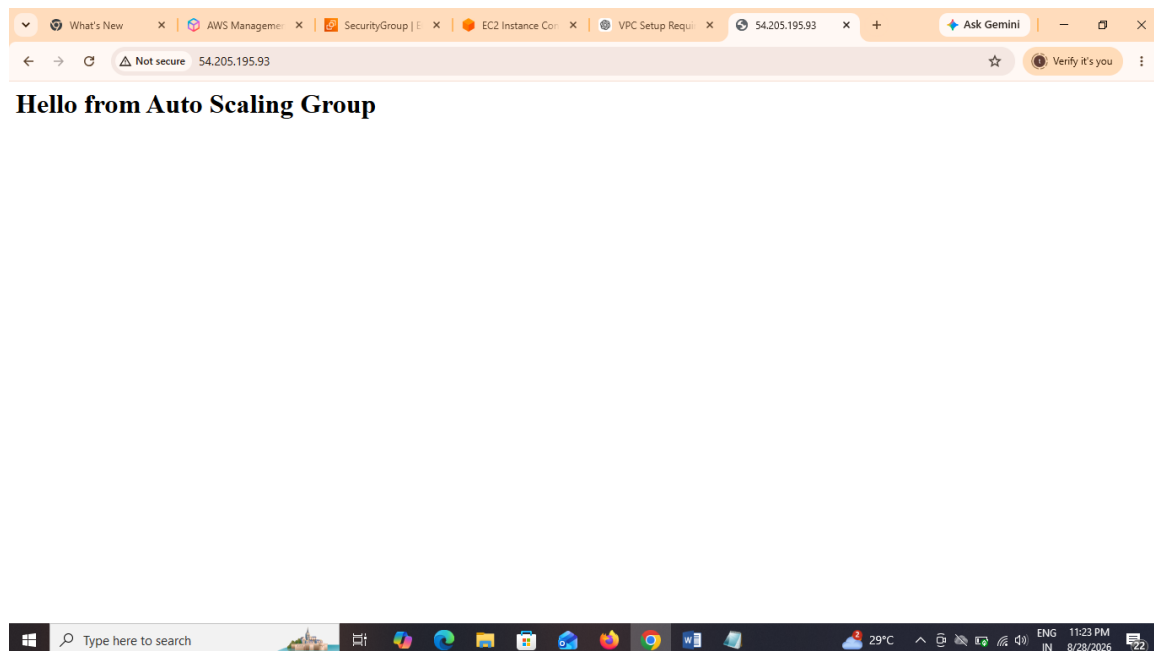


Figure 19 — The Apache-served web page, reachable over HTTP from the instance's public IP

The browser successfully loads the page served by Apache, confirming that the instance launched by the Auto Scaling Group is publicly reachable and serving traffic exactly as configured in the launch template's user data script.

5. Conclusion

This project successfully demonstrates how to host a website on AWS in a scalable and resilient way using core AWS networking and compute services. A custom VPC with public subnets in two Availability Zones was built from scratch, made internet-accessible through an Internet Gateway and route table, and used as the network layer for a Launch Template–driven Auto Scaling Group. The Auto Scaling Group automatically launched Ubuntu EC2 instances that self-configured into working Apache web servers via a user data script — with zero manual server setup.

The key benefit of this approach over a single manually configured EC2 instance is resilience and repeatability: if an instance fails or is terminated, the Auto Scaling Group automatically launches a replacement from the same Launch Template, and capacity can be increased simply by raising the desired/maximum instance count or attaching a scaling policy. This mirrors how production web applications are hosted on AWS in practice.

6. Key Learnings

- How a VPC, subnets, an Internet Gateway, and route tables work together to create public internet connectivity for EC2 instances.
- Why spreading subnets across multiple Availability Zones is critical for high availability.
- How Launch Templates standardize and version EC2 configuration for repeatable deployments.
- How user data scripts can fully automate server bootstrapping (package installation, service start, content deployment) with no manual login required.
- How Auto Scaling Groups use a Launch Template to maintain a desired number of healthy instances automatically, replacing failed ones without manual intervention.
- How security groups control inbound access (SSH for management, HTTP for public web traffic).